

Part 02

- 2- what's the difference between compiled and interpreted languages and in this way what about C#?

- A **compiled language** is a language where the source code is translated into machine code or an intermediate code by a compiler before the program runs. This usually makes the program faster during execution.
- An **interpreted language** is a language where the source code is executed by an interpreter at runtime, usually without producing a complete machine-code program beforehand.

What about C#?

- C# is considered a compiled language, but it uses an intermediate step. The C# compiler converts the source code into Intermediate Language (IL). Then, when the program runs, the JIT (Just-In-Time) compiler converts the IL into machine code that the computer's CPU can execute.

C# process:

- C# Source Code → C# Compiler → IL → JIT Compiler → Machine Code → Execution
- Therefore, C# is a **compiled language that uses JIT compilation at runtime**.

3- Compare between implicit, explicit, Convert and parse casting

Type	Meaning	Example	Notes
Implicit Casting	Conversion happens automatically by the compiler.	Int x =10; double y = x;	Usually safe because there is no data loss.
Explicit Casting	The programmer manually specifies the target type using (type).	double x = 10.5; int y = (int)x;	May cause data loss, such as losing the decimal part.
Parse	Converts a <code>string</code> into a specific type.	Int x = int.Parse("25");	Throws an exception if the string is not in a valid format.
Convert	Converts between different data types using the <code>Convert</code> class.	Int x = Convert.ToInt32("25");	Provides several conversion methods and can handle some <code>null</code> values.